

**Project PRISM: A Purple Team Laboratory for Detecting
Process Masquerading and OS Credential Dumping on Linux
via Kernel Audit Telemetry**

Gururam Subramanian (z5636559)
School of Computer Science and Engineering, UNSW Sydney
COMP6841: Security Engineering
30 June 2026

Abstract

Project PRISM is a self-contained Purple Team laboratory that pairs targeted adversary emulation with a custom-built behavioural detection engine. The project emulates two MITRE ATT&CK techniques — T1036 (Masquerading, sub-techniques .003 and .005) and T1003 (OS Credential Dumping, sub-technique .007 and direct shadow-file access) — using low-noise scripts that avoid dropping conventional malware artefacts on disk. On the defensive side, PRISM deploys Linux `auditd` kernel hooks defined in `prism.rules`, feeding a Python pipeline (`log_parser.py` → `correlator.py` → `alerter.py`) that converts raw `syscall` telemetry into structured, MITRE-mapped alerts. Two Palantir-format Alerting & Detection Strategy (ADS) documents formalise the detection logic, and a Purple Team Matrix cross-references every offensive action against the exact kernel event and detection rule it should trigger. In a controlled run on 1 July 2026, all four emulated offensive actions were detected within eight seconds of execution, and the exercise additionally surfaced three explicit blind spots — an audit-load timing gap, fileless execution via `memfd_create`, and un-correlated `ptrace` telemetry — that are documented rather than silently ignored. The report argues that the project’s central finding is not that the detections work, but that they work for a specific, name-able reason: kernel-level fields such as `exe` and `comm` cannot be forged from userspace the way file hashes and process names can, which is precisely why signature-based tooling misses both techniques. The report closes with a discussion of what a hardened, production iteration of PRISM would need to add, including eBPF-based sensors, `memfd_create` coverage, and Sigma-rule portability.

Introduction

Endpoint security has historically relied on two detection strategies: signature matching (does this file’s hash appear on a blocklist?) and static process-name monitoring (is `ps aux` showing anything unexpected?). Both strategies share a common weakness — they trust userspace-controlled data. A file hash changes the instant a binary is recompiled or padded; a process name displayed by `ps` is read from a field (`/proc/PID/comm`) that any process can rewrite via a single `syscall`.

Adversaries have exploited this gap for years, and two MITRE ATT&CK techniques capture it precisely: **T1036 (Masquerading)**, where a process is dressed up to look like something benign, and **T1003 (OS Credential Dumping)**, where credentials are lifted directly out of memory or out of the shadow password file rather than through a file-based store that a scanner might catch.

Project PRISM was scoped to answer a narrower, more falsifiable question than “can we detect malware”: *can a kernel-level telemetry source — `Linux auditd` — distinguish forged userspace identity from the underlying, kernel-enforced identity of a process, in real time, for two specific sub-techniques?* This framing shaped every design decision in the project. Rather than building a general-purpose EDR agent, PRISM restricts itself to two adversary behaviours it can emulate precisely, instruments exactly the syscalls those behaviours touch, and validates the result against a fixed, repeatable run rather than a subjective “it looked like it worked.”

The project’s scope covers four components, mirroring the standard purple-team structure: a **Red Team** emulation suite (`red/`) that performs the two techniques against a dummy target inside an isolated VM; a **Blue Team** detection engine (`blue/`) that hooks the relevant syscalls and correlates the resulting audit trail; a pair of **ADS documents** (`ads/`) that formalise each detection rule in the Palantir framework used across the industry; and a **Purple Matrix** (`matrix/purple_matrix.md`) that maps every offensive action to the telemetry it produces and the alert it should fire, including an explicit accounting of what is *not* detected.

The objectives set out in the Week 3 proposal were threefold: (1) build low-noise Red Team scripts that emulate T1036 and T1003 without relying on off-the-shelf offensive tooling that would itself be signature-detectable; (2) build a Blue Team pipeline capable of correlating raw kernel audit events into MITRE-mapped alerts with an explicitly managed false-positive rate; and (3) close the loop by running the two halves against each other and documenting, honestly, where detection succeeds and where it does not. All three objectives were met, and Sections 4 through 6 of this report walk through how.

Background & Related Work

MITRE ATT&CK as a Common Vocabulary

The MITRE ATT&CK framework organises adversary behaviour into tactics (the “why,” e.g. Defense Evasion, Credential Access) and techniques (the “how”) (MITRE Corporation, 2026a). PRISM targets two techniques that sit in different tactic categories but share a common root cause: both exploit the gap between what a process *claims* to be, at the userspace layer, and what the kernel actually enforces.

- **T1036 — Masquerading**, and specifically **T1036.003** (renaming or relocating a system utility so its `argv[0]`/file name mimics a trusted binary) and **T1036.005** (rewriting a running process’s kernel-visible name via `prctl(PR_SET_NAME)` so tools like `ps` and `top` display a fake, trusted name) (MITRE Corporation, 2026a). Both sub-techniques fall under the Defense Evasion tactic — the adversary is not trying to gain new access, but to avoid being noticed by anyone glancing at a process list.
- **T1003 — OS Credential Dumping**, and specifically **T1003.007** (reading a target process’s virtual memory directly through `/proc/[pid]/mem`) (MITRE Corporation, 2026b) alongside the broader technique of reading `/etc/shadow` for offline password cracking (MITRE Corporation, 2026c). This falls under Credential Access, and is the Linux analogue of LSASS memory dumping on Windows: rather than touching a file-based credential store that file-integrity monitoring watches, the adversary pulls secrets straight out of RAM.

The Palantir Alerting & Detection Strategy Framework

PRISM’s detection documentation follows the Palantir ADS framework, an open-source structure for writing a single, self-contained detection specification (Palantir Technologies, n.d.). Each ADS answers a fixed set of questions in order — what is the goal, where does the technique sit in ATT&CK, what is the detection strategy in the abstract, what is the underlying OS mechanism, why do standard tools miss it, what are the known blind spots and false positives, how is the detection validated, what severity does it warrant, and what is the analyst response playbook. The

value of this structure is that it forces the author to commit, in writing, to the technique’s blind spots *before* the detection is presented as a success — which is why Section 6 of this report and the ADS documents in Appendix B both contain explicit “Blind Spots and Assumptions” sections rather than only reporting the true positives.

Linux Audit as a Kernel-Level Data Source

`auditd` is the userspace daemon for the Linux Auditing Framework, a kernel subsystem that can log arbitrary syscalls, file-watch events, and their associated process context (UID, PID, PPID, `exe`, `comm`, arguments) to `/var/log/audit/audit.log` (Linux man-pages project, n.d.-b). Its relevance to PRISM is that the fields it exposes are sourced from the kernel’s own task struct rather than from anything a process can pass to a monitoring agent voluntarily. This matters specifically for T1036.005: the `comm` field can be rewritten by `prctl(PR_SET_NAME)` (Linux man-pages project, n.d.-c), but `exe` — a symlink maintained by the kernel at `/proc/PID/exe` (Linux man-pages project, n.d.-d) — cannot be. The entire T1036 detection strategy rests on comparing these two fields and treating any mismatch as suspicious.

Prior Work and Positioning

Commercial EDR products (e.g. Falcon, Defender for Endpoint, Elastic Security) implement broadly similar behavioural rules to the ones built here, but as closed, tuned rule sets layered over much larger telemetry pipelines. Open validation frameworks such as Red Canary’s Atomic Red Team (Red Canary, n.d.) provide scripted technique emulations similar in spirit to PRISM’s `red/` scripts, but are technique-generic rather than paired with a bespoke, from-scratch detection engine. PRISM’s contribution is not a novel detection idea — the `exe/comm` mismatch and the `/proc/[pid]/mem` open-event heuristic are both well-documented in industry literature — but a small, fully auditable, end-to-end implementation where every layer (the attack, the kernel hook, the parser, the correlator, and the alert) was built and can be inspected, rather than treated as a vendor black box.

Environment & Architecture

PRISM runs entirely inside an isolated Ubuntu 22.04 LTS virtual machine, provisioned once via `bootstrap_env.sh`, which installs `auditd` (≥ 2.8), `gcc`, and Python (≥ 3.10), creates a low-privilege `prism_test` user, and deploys a dummy target process (`dummy_cred_holder.sh`) that holds a planted, fake credential string in its heap for the T1003 simulation to locate. No production credentials, hostnames, or external network paths are used anywhere in the project; the shadow-file and memory-scraping targets are both synthetic.

The repository is organised into four functionally separate directories that mirror the Red/Blue/ADS/Matrix structure described above:

```
project-prism/
|-- red/                                # Offensive simulation suite
|   |-- t1036_masquerade/              # rename_exec.sh, prctl_spoof.c, run_t1036.sh
|   |-- t1003_cred_dump/              # proc_mem_scan.py, shadow_access.sh, run_t1003.sh
|   '-- red_runner.sh                 # Top-level attack harness
|-- blue/                              # Defensive monitoring engine
|   |-- auditd/                       # prism.rules, install_hooks.sh
|   |-- parser/                       # log_parser.py, correlator.py, alerter.py, models.py
|   '-- engine.sh
|   '-- requirements.txt
|-- ads/                               # Palantir-format ADS documents
|-- matrix/purple_matrix.md           # Red → Telemetry → Detection mapping
|-- reports/                         # proposal.md, final_report.md
|-- tests/                           # validate_t1036.sh, validate_t1003.sh
|-- logs/                            # Runtime artefacts (git-ignored)
'-- bootstrap_env.sh
```

Kernel hooks are installed by copying `prism.rules` into `/etc/audit/rules.d/` and reloading with `augenrules --load` (Linux man-pages project, n.d.-a), which requires root. The Blue Team pipeline itself runs unprivileged once the audit log is readable: `blue/engine.sh` orchestrates `log_parser.py` (raw log → normalised JSON), `correlator.py` (normalised JSON → candidate detections), and `alerter.py` (candidate detections → formatted alert output), writing intermediate artefacts to `logs/parsed/parsed_telemetry.json` and `logs/parsed/detections.json`. This three-stage separation — parse, correlate, alert — was a deliberate architectural choice so that correlation logic (the part most likely to need tuning as false positives are discovered) is isolated from both the raw log format and the alert presentation layer.

Offensive Module (Red Team)

T1036 — Masquerading

The T1036 simulation targets both in-scope sub-techniques with separate, minimal scripts rather than a single combined tool, so that each technique’s kernel footprint can be observed in isolation.

T1036.003 (`rename_exec.sh`) copies a legitimate, harmless system binary (`/bin/sleep`) into a writable temporary directory under a name chosen to look like a systemd component (`/tmp/systemd-helper`), marks it executable, and runs it. This is deliberately not a novel technique — it is the textbook “living off the land” move of relocating a trusted binary rather than writing new code — chosen precisely because it is well understood and therefore a fair test of whether the detection layer catches relocation rather than payload content. The design decision to reuse `/bin/sleep` rather than write a custom binary was intentional: it demonstrates that the detection strategy does not depend on the payload being unusual in any way, only on where it was executed from.

T1036.005 (`prctl_spoof.c`) is a small C program that calls `prctl(PR_SET_NAME, "kworker/u4:2", ...)` immediately after starting, rewriting its own kernel-visible thread name to mimic a legitimate kernel worker thread. This is a more surgical technique than T1036.003: no file relocation is needed, and to a defender running `ps` or reading `/proc/PID/comm`, the process is indistinguishable from a real kernel worker. The only field that still tells the truth is `/proc/PID/exe`, which continues to point at the real binary path regardless of what `comm` claims.

Evidence captured from the 1 July 2026 run confirms both mechanisms worked as designed. The raw audit delta for T1036.003 shows `exe=/tmp/systemd-helper` paired with `comm=systemd-helper` — a path outside any legitimate install location, self-labelled with a name designed to look benign. The T1036.005 delta shows `syscall=158` (the `prctl` syscall number) with argument `a0=0xf` (option 15, `PR_SET_NAME`), and — critically — an `exe` field of `/tmp/prism_prctl_spoof` sitting alongside a `comm` field reading `kworker/u4:2`. That

mismatch between what the kernel knows the binary to be and what the process has told userspace tools to display is the entire signal the Blue Team engine relies on.

T1003 — OS Credential Dumping

T1003.007 (`proc_mem_scan.py`) performs the canonical `/proc` memory-scraping workflow without dropping any dedicated dumping tool on disk: it enumerates `/proc/` to find the dummy target's PID, reads `/proc/[pid]/maps` to identify readable memory regions (heap, stack, mapped files) (Linux man-pages project, n.d.-d), opens `/proc/[pid]/mem` with `O_RDONLY`, seeks to an offset inside a readable region defined by the kernel's syscall table (Torvalds & Linux Kernel Contributors, n.d.), and reads. In the validated run, the script located the dummy target at PID 223290 and extracted the planted string `password=DUMMY_LAB_SECRET_PRISM_2024` directly from the process's `[heap]` region at address `0x55f71ed91000`. No credential was written to disk at any point, and no `ptrace` call was made — the entire extraction happened through ordinary, individually-innocuous `open/read` syscalls, which is exactly what makes this class of attack hard for file-based or signature-based tools to see.

Shadow file access (`shadow_access.sh`) simply confirms read access to `/etc/shadow`, simulating the second half of a credential-dumping playbook: after (or instead of) scraping live process memory, an adversary with sufficient privilege reads the shadow file directly to obtain hashed passwords for offline cracking with tools such as `hashcat` or `john`. This was included specifically because it is a *different* attack surface from `/proc/mem` scraping — it requires no memory-layout knowledge at all, only file permissions — and PRISM's Blue Team engine needed to be tested against both.

Both offensive scripts were run inside `red/red_runner.sh`, which sequences the T1036 and T1003 harnesses and captures the resulting audit deltas to `logs/raw/` for later correlation, keeping the offensive and defensive halves of the exercise cleanly separated in time and in the filesystem.

Defensive Module (Blue Team)

Auditd Rule Design Rationale

`prism.rules` (reproduced in full in Appendix A) hooks exactly the syscalls the Red Team scripts exercise, and no more — a deliberate scoping decision to keep the audit log volume manageable and the correlator’s job tractable. Five rule groups are defined:

1. `execve` events filtered to `/tmp`, `/dev/shm`, and `/var/tmp` (T1036.003) — execution from these paths is rare enough for legitimate system processes that it is treated as a strong prior signal on its own.
2. All `prctl` syscall events (T1036.005) — this rule is intentionally broad (it cannot be filtered by argument at the audit-rule level) and relies on the correlator, not the kernel rule, to narrow down to `PR_SET_NAME` calls with a suspicious `exe/comm` mismatch.
3. `open/openat` events under `/proc` (T1003.007) — again broad at the kernel-rule level, narrowed downstream to `/proc/[pid]/mem` and `/proc/[pid]/maps` paths specifically, with `/proc/self/*` explicitly excluded to suppress JVM, Go-runtime, and self-inspecting-process noise.
4. `ptrace` syscall events (T1003, alternate vector) — included as an audit key (`prism_t1003_ptrace`) but, as documented honestly in Section 6, not yet wired into the correlator, since debuggers such as `gdb` and `strace` would otherwise generate a high false-positive rate without additional context the current correlator does not yet compute.
5. File watches on `/etc/shadow` and `/etc/gshadow` (T1003 general) — a `-w` watch rather than a syscall filter, since these are two fixed, well-known paths rather than a pattern that needs syscall-level matching.

The recurring design principle across all five rule groups is to push filtering *complexity* into the Python correlator and keep the kernel-level `auditd` rules broad and cheap to evaluate — kernel-side filtering has real performance cost at high syscall volume, whereas Python-side filtering can

use richer logic (whitelists, path-prefix checks, parent-process lookups) without touching the hot path.

Parser and Correlation Architecture

`log_parser.py` is the first stage of the pipeline: it consumes raw `auditd` log lines (which are emitted as loosely-structured `key=value` text, often split across multiple related `SYSCALL` and `PATH` records sharing one audit event ID) and normalises them into a consistent JSON schema, defined in `models.py` as `AuditEvent`, `Transaction`, and `Alert` data classes. This normalisation step exists so that neither the correlator nor any future consumer of the data needs to re-parse `auditd`'s native text format.

`correlator.py` implements the two behavioural detection functions referenced throughout this report: `detect_t1036_masquerading`, which compares each event's `exe` and `comm` fields and checks execution paths against the `/tmp`, `/dev/shm`, `/var/tmp` set, and `detect_t1003_credential_dumping`, which filters `/proc open` events down to `mem/maps` paths (excluding `/proc/self/*`) and flags any `/etc/shadow` or `/etc/gshadow` access from a binary not present in a hardcoded whitelist of legitimate authentication tooling (`sudo`, `su`, `passwd`, `unix_chkpwd`). Both functions also implement the T1036 false-positive suppression list described below, filtering out `prctl` calls originating from interpreters and runtimes (`python3`, `java`, `go`) that legitimately rename their own worker threads for unrelated reasons.

`alerter.py` is the final stage, formatting confirmed detections into the alert schema written to `logs/parsed/detections.json` — each alert carries the MITRE technique ID, a human-readable title, severity, and the underlying evidence fields (`pid`, `exe`, `comm`, `path`), so that an analyst reading the JSON output does not need to cross-reference the raw audit log to understand what fired and why.

Per-Rule Detection Logic, Strategy, and False-Positive Risk

Rule	Strategy	FP Risk
T1036.003 — temp-path execution	Flag any <code>execve</code> where <code>exe</code> resolves under <code>/tmp</code> , <code>/dev/shm</code> , or <code>/var/tmp</code>	Medium — CI/CD pipelines and package-manager postinstall scripts legitimately execute from <code>/tmp</code> ; suppressed by filtering on parent process (<code>ppid</code> resolving to <code>apt</code> , <code>dpkg</code> , <code>cmake</code> , <code>make</code>)
T1036.005 — <code>prctl</code> name spoof	Flag any <code>prctl(PR_SET_NAME)</code> where <code>comm</code> (post-call) mimics a known system/kernel name while <code>exe</code> sits outside expected system paths	Medium — Python, Java, and Go runtimes rename worker threads routinely; suppressed by an <code>exe</code> -substring whitelist (<code>KNOWN_PRCTL_CALLERS</code> in <code>correlator.py</code>)
T1003.007 — <code>/proc/[pid]/mem</code> access	Flag <code>open/openat</code> on any <code>/proc/[pid]/mem</code> or <code>/proc/[pid]/maps</code> path not under <code>/proc/self/</code>	Low, once self-reads are excluded — remaining noise sources are diagnostic tools (<code>ps</code> , <code>top</code> , <code>htop</code>), suppressed via a standard-binary whitelist
T1003 — <code>shadow/gshadow</code> access	Flag any <code>open</code> on <code>/etc/shadow</code> or <code>/etc/gshadow</code> from an <code>exe</code> outside the authentication-tooling whitelist	Low — the whitelist (<code>sudo</code> , <code>su</code> , <code>passwd</code> , <code>unix_chkpwd</code>) covers essentially all legitimate readers on a standard Ubuntu install
T1003 — <code>ptrace</code>	Audit key exists (<code>prism_t1003_ptrace</code>)	Not yet correlated — see Section 6.3

All four “live” rules were tuned against one concrete false-positive incident discovered during validation: an early version of the correlator generated 108 spurious T1003.007 alerts from processes reading their own `/proc/self/maps`, which was fixed by adding the `/proc/self/*` exclusion described above. That fix, and the fact that it was needed at all, is discussed further in Section 6.

Purple Team Feedback Loop

Integration Test Results

The closed-loop validation run executed on 1 July 2026 in the isolated lab VM produced the results summarised below (full detail in Appendix C). Four distinct offensive actions were emulated, and all four were detected by the correlator within eight seconds of execution — the time between the Red Team script issuing its syscall and the corresponding alert appearing in `logs/parsed/detections.json`:

1. **T1036.003** — `rename_exec.sh` copying and executing `/bin/sleep` as `/tmp/systemd-helper` → detected via `detect_t1036_masquerading`, alert title *“Process in temp directory claimed legitimate system name.”*
2. **T1036.005** — `prctl_spoof.c` renaming itself to `kworker/u4:2` → detected via the same correlator function, alert title *“Process explicitly renamed to system thread via prctl.”*
3. **T1003.007** — `proc_mem_scan.py` reading the dummy target’s heap through `/proc/223290/mem` → detected via `detect_t1003_credential_dumping`, alert title *“Unauthorized /proc memory access,”* with the planted secret confirmed extracted.
4. **T1003 (shadow)** — `shadow_access.sh` reading `/etc/shadow` from a non-whitelisted bash process → detected via the same function, alert title *“Unauthorized shadow file access.”*

The following sample JSON objects are representative of each alert type as written to `logs/parsed/detections.json` by `alerter.py`:

```
[ ] [ { "rule": "T1036.005-Masquerade", "severity": "HIGH",
"description": "Process in temp directory claimed legitimate system
name: exe=/tmp/systemd-helper | comm=systemd-helper", "audit_id":
"5492" }, { "rule": "T1036.005-Masquerade-Prctl", "severity":
"HIGH", "description": "Process explicitly renamed to system
```

```
thread via prctl: exe=/tmp/prism_prctl_spoof | comm=kworker/u4:2",
"audit_id": "5501" }, { "rule": "T1003.007-CredentialDump",
"severity": "CRITICAL", "description": "Unauthorized /proc memory
access: exe=/usr/bin/python3 | target=/proc/223290/mem", "audit_id":
"6001" }, { "rule": "T1003-ShadowAccess", "severity": "CRITICAL",
"description": "Unauthorized shadow file access: exe=/usr/bin/bash |
target=/etc/shadow", "audit_id": "5900" } ]
```

A fifth row in the matrix (execution from `/dev/shm` rather than `/tmp`) is covered by the same rule and correlator branch as row 1 but was not separately exercised in the live run; it is recorded as covered by construction rather than by direct test evidence, and this distinction is preserved honestly in the matrix rather than conflated with the four directly observed detections.

Across the full run, the engine emitted 112 raw alerts in total, of which only 4 were true positives matching the intended simulation. The remaining 108 were the `/proc/self/maps` false-positive cluster described in Section 5.3, generated primarily by the Python interpreter running `proc_mem_scan.py` itself inspecting its own memory layout as an ordinary part of interpreter operation. This is reported as a finding, not hidden as noise: it demonstrates concretely why a `/proc-open`-based detection rule is unusable in production without a self-read exclusion, and it is the single clearest piece of evidence in the whole project that naive kernel telemetry still needs behavioural tuning before it becomes an actionable alert stream.

Purple Matrix

The full Red $\rightarrow$ Telemetry $\rightarrow$ Detection mapping is provided in Appendix C. Its structure — one row per offensive action, with columns for the exact kernel event, the `auditd` key, the raw fields observed, the correlator function and alert title, a detected/not-detected verdict, observed latency, and a notes column explaining *why* — is deliberately more granular than a simple pass/fail table, because the purpose of a purple matrix is to make the causal chain from attack to alert auditable by a third party, not merely to report a success rate.

Remaining Blind Spots

Three gaps are documented in the matrix and are treated as first-class findings of the project rather than omissions to be quietly fixed later:

- **Early-persistence timing gap:** if an adversary calls `prctl(PR_SET_NAME)` before `auditd`'s rules are loaded at boot — for example, from a very early-boot persistence mechanism — the event is simply never captured, because there is no rule in place yet to catch it. The documented mitigation is to load rules in immutable mode (`-e 2`) via `augenrules` from an early-boot `systemd` unit, closing the window rather than trying to detect activity retroactively.
- **Fileless execution via `memfd_create`:** an adversary who creates an anonymous, in-memory file descriptor with `memfd_create` and executes it via `fexecve()` never triggers a conventional `execve` event against a real filesystem path, so the T1036.003 rule — which filters on `dir=/tmp` and similar — never fires. This is arguably the most serious blind spot in the current ruleset, since it defeats the entire “flag execution from suspicious paths” strategy at the root. The documented remediation is to add a dedicated audit rule for `memfd_create` (syscall 319) and correlate it with a subsequent `fexecve` from the same PID.
- **Un-correlated `ptrace` telemetry:** the `prism_t1003_ptrace` audit key exists and captures every `ptrace()` call, but the correlator does not yet turn these into alerts, because legitimate debugging tools (`gdb`, `strace`, `gcore`) generate `ptrace` calls constantly and a naive rule would produce an unusable false-positive rate. This is recorded as a partial detection (row 8 in the matrix) rather than a full gap, since the telemetry is being captured — the missing piece is correlation logic, not data collection.

Each blind spot is paired with a concrete future mitigation rather than left as an open question, which is consistent with the Palantir ADS philosophy of documenting detection boundaries as precisely as detection successes.

Analysis

Why do standard signature-based tools miss these techniques? Signature-based antivirus and EDR tooling primarily key off two things: file hashes and, for behavioural products, process names as reported by the OS’s normal userspace APIs. T1036.003 defeats hash-based detection trivially, because copying `/bin/sleep` to a new path changes nothing about the file’s hash or content — a hash-based scanner only catches it if it re-scans every file on disk on every execution, which most tools do not do continuously. T1036.005 goes a step further and defeats *name*-based detection specifically, because `prctl(PR_SET_NAME)` changes the very field (`comm`) that tools like `ps`, `top`, and naive `/proc/PID/comm` readers use to identify a process; a monitoring tool watching only that field sees exactly what the adversary wants it to see. T1003.007 defeats both categories simultaneously: there is no malicious binary to hash (the scraping was done with an ordinary, signed `python3` interpreter), and there is no anomalous file write to trigger file-integrity monitoring, since reading `/proc/[pid]/mem` produces no persistent artefact at all.

How does kernel telemetry surface behavioural anomalies that userspace tools cannot?

The consistent thread across all four detections in this project is that `auditd` reads process identity fields directly from the kernel’s task struct rather than from any userspace-facing API a process could influence. `/proc/PID/exe` is a kernel-maintained symlink to the file that was actually `exec()`’d; a process cannot rewrite it, no matter what it does to its own `comm` field or `argv[0]`. This is why the T1036 correlator’s core logic is a *comparison* — `comm` vs. `basename(exe)` — rather than a check against either field alone: the adversary can forge one side of that comparison, but not both, and the mismatch itself is the signal. The same principle underlies the T1003.007 rule in a slightly different form: it is not that reading `/proc/[pid]/mem` is inherently malicious (legitimate debuggers and runtimes do it constantly, hence the `/proc/self` exclusion and the 108-alert false-positive incident recorded in Section 6.1), but that the kernel event itself — an `open()` syscall on a specific, sensitive path — is visible and loggable regardless of what tool performed it or how it disguised its intent.

What properties of the Palantir ADS framework make it resilient? The framework’s most

useful property, observed directly during this project, is that it structurally forces a “Blind Spots and Assumptions” section before a detection can be presented as validated. This is not a stylistic choice — it changes what gets built. The `memfd_create` gap documented in Section 6.3 was identified precisely because writing the ADS document for T1036 required explicitly reasoning about “why standard defenses miss this” and, by extension, what *else* might defeat the new defence in the same way. A detection strategy written as a flat “here is our rule” statement has no natural place to record that reasoning; the ADS format’s fixed sections (Goal → Categorization → Strategy Abstract → Technical Context → Blind Spots → False Positives → Validation → Priority → Response) make the omission of that analysis visibly incomplete rather than simply absent.

Conclusions

Project PRISM set out to test a narrow, falsifiable claim — that kernel-sourced identity fields can catch forged userspace identity for two specific ATT&CK techniques, in real time — and the validated run supports that claim for the four offensive actions directly tested. The project’s more durable lessons, however, sit alongside that headline result rather than in it.

Lessons learned: First, a detection rule that looks correct on paper still needs to be run against realistic background activity before it can be trusted; the 108 false-positive `/proc/self` alerts generated by nothing more exotic than the Python interpreter running the Red Team’s own scraping script were not predicted at design time and were only found by actually executing the loop. Second, documenting blind spots at design time (as the ADS framework requires) surfaces gaps — like the `memfd_create` fileless-execution path — that would otherwise only be discovered adversarially, after a real bypass. Third, kernel-level telemetry is a necessary but not sufficient condition for good detection: `auditd` correctly captured every event needed for all four detections, but the correlation logic that turns “an event happened” into “this event means something” is where the actual analytic work, and the actual risk of both false positives and false negatives, lives.

What the next iteration would add: Three concrete extensions follow directly from the blind spots recorded in Section 6.3. An **eBPF-based sensor** would close the early-persistence timing

gap more robustly than immutable `auditd` rules alone, since eBPF programs can be attached earlier in the boot sequence and are harder to disable without triggering their own tamper alerts. A **`memfd_create/fexecve` correlation rule** is the most urgent addition, since fileless execution is currently a complete bypass of the T1036.003 strategy rather than a partial gap. Finally, **porting the correlator’s detection logic to Sigma rules** — a vendor-neutral, YAML-based detection format — would let PRISM’s two ADS documents be consumed by SIEM platforms beyond this project’s own Python pipeline, turning a bespoke lab exercise into something portable to a real SOC toolchain, which was one of the original motivations named in the Week 3 proposal’s definition of done.

References

- Linux man-pages project. (n.d.-a). *auditctl(8) — Audit rule management*. Retrieved July 25, 2026, from <https://man7.org/linux/man-pages/man8/auditctl.8.html>
- Linux man-pages project. (n.d.-b). *auditd(8) — Linux audit daemon*. Retrieved July 25, 2026, from <https://man7.org/linux/man-pages/man8/auditd.8.html>
- Linux man-pages project. (n.d.-c). *prctl(2) — Operations on a process*. Retrieved July 25, 2026, from <https://man7.org/linux/man-pages/man2/prctl.2.html>
- Linux man-pages project. (n.d.-d). *proc(5) — Process information pseudo-filesystem*. Retrieved July 25, 2026, from <https://man7.org/linux/man-pages/man5/proc.5.html>
- MITRE Corporation. (2026a). *Masquerading, Technique T1036*. MITRE ATT&CK. <https://attack.mitre.org/techniques/T1036/>
- MITRE Corporation. (2026b). *OS credential dumping: /proc filesystem, Technique T1003.007*. MITRE ATT&CK. <https://attack.mitre.org/techniques/T1003/007/>
- MITRE Corporation. (2026c). *OS credential dumping, Technique T1003*. MITRE ATT&CK. <https://attack.mitre.org/techniques/T1003/>
- Palantir Technologies. (n.d.). *Alerting and detection strategy (ADS) framework* [Computer software]. GitHub. <https://github.com/palantir/alerting-detection-strategy-framework>

Red Canary. (n.d.). *Atomic Red Team* [Computer software]. GitHub.

<https://github.com/redcanaryco/atomic-red-team>

Torvalds, L., & Linux Kernel Contributors. (n.d.). *Linux kernel x86-64 syscall table*. GitHub.

https://github.com/torvalds/linux/blob/master/arch/x86/entry/syscalls/syscall_64.tbl

Appendix A — Full `prism.rules` Auditd Ruleset

```
# PRISM System Auditing Rules
# Project PRISM -- Purple-Red Intelligence & Surveillance Monitor
# Author: Gururam Subramanian (z5636559) -- COMP6841 Security Engineering
# Remove any existing rules first
-D

# Buffer Size -- large enough for high-volume runs
-b 8192

# =====
# MITRE T1036.003: Rename System Utilities
# Detects execve() called from writable temp directories.
# Key: exe path is in /tmp, /dev/shm, or /var/tmp -- not a
# legitimate install location for system binaries.
# =====
-a always,exit -F arch=b64 -S execve -F dir=/tmp -k prism_t1036_exec_tmp
-a always,exit -F arch=b64 -S execve -F dir=/dev/shm -k prism_t1036_exec_shmem
-a always,exit -F arch=b64 -S execve -F dir=/var/tmp -k prism_t1036_exec_vartmp

# =====
# MITRE T1036.005: Match Legitimate Name or Location
# Detects prctl(PR_SET_NAME) calls -- the syscall used to
# rename a process thread to mimic kernel worker names.
# syscall 158, option a0=0xf (15 = PR_SET_NAME).
# =====
-a always,exit -F arch=b64 -S prctl -k prism_t1036_prctl

# =====
# MITRE T1003.007: /proc Filesystem Memory Access
# Hooks open() and openat() on anything under /proc.
# Correlator narrows to /proc/[pid]/mem and /proc/[pid]/maps.
# =====
-a always,exit -F arch=b64 -S open,openat -F dir=/proc -k prism_t1003_proc_open

# =====
# MITRE T1003: ptrace-based Memory Access
# Alternative credential dump vector via ptrace().
# Higher FP risk (debuggers); correlated separately.
# =====
-a always,exit -F arch=b64 -S ptrace -k prism_t1003_ptrace

# =====
# MITRE T1003: Shadow File Access
# Watches reads and writes to /etc/shadow and /etc/gshadow.
# Any non-whitelisted binary opening these files is CRITICAL.
# =====
-w /etc/shadow -p rwa -k prism_t1003_shadow
-w /etc/gshadow -p rwa -k prism_t1003_gshadow
```

Appendix B — Full ADS Documents

ADS: Process Masquerading on Linux Endpoints (T1036 / T1036.003 / T1036.005)

Status: Production-Ready (PRISM Lab) | **Priority:** HIGH | **Last Updated:** 2026-06-30

Goal: Detect adversary attempts to disguise malicious processes by either executing a renamed copy of a legitimate system binary from a non-standard path (`/tmp`, `/dev/shm`, `/var/tmp`), or spoofing a process's reported name via `prctl(PR_SET_NAME)`.

Categorization: Defense Evasion / T1036 — Masquerading. Sub-techniques in scope: T1036.003 (Rename System Utilities) and T1036.005 (Match Legitimate Name or Location). Data source: Linux auditd (`execve`, `prctl` syscall events).

Strategy Abstract: Rule 1 subscribes to `execve` events where `exe` resolves under a writable temp path — a strong indicator on its own, since legitimate system processes essentially never originate there. Rule 2 subscribes to `prctl` events with `option=15` (`PR_SET_NAME`); the definitive signal is the mismatch between the kernel-set `exe` field and the userspace-writable `comm` field after the call.

Technical Context: Every Linux process exposes two distinct name fields — `/proc/PID/comm` (writable via `prctl`) and `/proc/PID/exe` (a kernel-maintained symlink, not forgeable from userspace). `auditd` logs `execve()` as `SYSCALL type=59` and `prctl()` as `SYSCALL type=158` with `a0` encoding the option. Standard defences miss this because signature/hash-based AV does not catch a renamed copy of a legitimate binary, process-name monitors are trivially defeated by `prctl(PR_SET_NAME)`, and most tools fail to correlate `comm` against `exe` as a pair.

Blind Spots and Assumptions: (1) legitimate CI/CD and package-manager temp-path execution must be suppressed by parent-process filtering; (2) `prctl` calls issued before `auditd` rules load at boot are missed entirely; (3) fully in-memory execution via `memfd_create+execve()` bypasses this rule and would need a dedicated `memfd_create` (syscall 319) hook; (4) container workloads may need path scoping to avoid false positives from

legitimate `/tmp` use; (5) `prctl PR_SET_NAME` audit logging requires kernel ≥ 4.15 and `auditd` ≥ 2.8 .

False Positives: Python/Java/Go runtime threading triggers T1036.005 and is suppressed by an `exe-substring` whitelist; `cmake/make` test runners and package-manager postinst scripts trigger T1036.003 and are suppressed by parent-process (`ppid`) checks. All suppression logic lives in `blue/parser/correlator.py`.

Validation: T1036.003 — `copy /bin/sleep to /tmp/systemd-helper`, execute, confirm alert fires and `/proc/$!/exe` resolves to the temp path. T1036.005 — compile and run `prctl_spoof.c`, confirm `/proc/$!/comm` reads `kworker/u4:2` while `/proc/$!/exe` reveals the true temp-path binary. Automated: `bash tests/validate_t1036.sh` (expected exit 0).

Priority: HIGH for all four listed conditions (temp-path execution regardless of UID; `prctl` spoofing combined with a temp-path `exe`), stepping down to MEDIUM where `exe` sits in a normal path but `comm` still spoofs a system name.

Response: Triage (<5 min) by noting `pid/exe/ppid`; confirm (<10 min) by comparing `comm` against `basename(exe)`; preserve process tree, open files, and network connections; isolate by terminating and capturing a memory snapshot if forensics are required; hunt for additional masqueraded children from the same parent PID.

ADS: OS Credential Dumping via /proc Filesystem and Shadow File Access (T1003 / T1003.007)

Status: Production-Ready (PRISM Lab) | **Priority:** CRITICAL | **Last Updated:** 2026-06-30

Goal: Detect adversaries reading process memory via `/proc/[pid]/mem` to extract in-memory credentials without triggering file-based detection, and detect unauthorised reads of `/etc/shadow//etc/gshadow`.

Categorization: Credential Access / T1003 — OS Credential Dumping. Sub-technique in scope: T1003.007 (`/proc` Filesystem), plus general shadow-file access. Data source: Linux `auditd` (`open/openat` on `/proc/*/mem` and `/etc/shadow`).

Strategy Abstract: Rule 1 subscribes to `open/openat` events matching `/proc/[pid]/mem` or `/proc/[pid]/maps`, the Linux equivalent of LSASS dumping, with `/proc/self/*` reads filtered out to remove JVM/Go/sanitiser noise. Rule 2 subscribes to `open/openat` on the shadow files from any exe outside a hardcoded authentication-tooling whitelist.

Technical Context: `/proc/[pid]/mem` grants read access to a process's virtual address space to any reader sharing its UID (with `ptrace` permission) or running as root. The documented lab result confirmed `proc_mem_scan.py` located a dummy target and extracted a planted secret from its `[heap]` region. Shadow passwords exist specifically to move hashed credentials out of world-readable `/etc/passwd`; a read of `/etc/shadow` produces no write and so triggers no file-integrity-monitoring alert, and the reader need not be a purpose-built dumping tool — an ordinary `bash` or `python3` process suffices.

Blind Spots and Assumptions: (1) kernel-module or eBPF-based memory reads bypass `open()` entirely and are invisible to this rule; (2) `ptrace`-based readers (`gdb`, `gcore`) use a different syscall, covered by a separate, not-yet-correlated audit key; (3) self-reads of `/proc/self/mem` are legitimate and filtered; (4) container escape to a host PID namespace would need container-aware filtering; (5) some PAM modules legitimately read shadow and require whitelist maintenance as new modules are introduced.

False Positives: `sudo/su/passwd` and `unix_chkpwd` are whitelisted for shadow access; `/proc/self/mem` reads (JVM, Go, debuggers on themselves) and `systemd` reads of `/proc/pressure/*` are filtered for the proc-memory rule; standard diagnostic tools (`ps`, `top`, `htop`) are whitelisted.

Validation: Start the dummy credential holder, run `proc_mem_scan.py` as root, confirm the planted secret is extracted and the alert fires; run `shadow_access.sh` as root and confirm the shadow-access alert fires. Automated: `sudo bash tests/validate_t1003.sh` (expected exit 0).

Priority: CRITICAL for `/proc/[pid]/mem` access regardless of the reading UID, and

for shadow-file reads by a non-whitelisted binary running as root; HIGH for a non-root, non-whitelisted shadow read or a `ptrace()` call across UIDs.

Response: Triage (<5 min) by identifying reader and target PID; confirm (<10 min) whether the reader is an expected debugger or security tool; assess the blast radius of the target process (SSH agent, browser, password manager); preserve the reading process's cmdline/exe/maps and snapshot if still running; escalate shadow-file access to a forced password rotation across all local accounts; hunt for lateral movement via SSH auth logs.

Appendix C — Purple Team Matrix

Run evidence collected 2026-07-01 in an isolated Ubuntu 22.04 LTS VM.

ID	Technique	Offensive Action	Kernel Event	Audit Key	Detection Rule	Alert Title	Detected?	Latency
1	T1036.003	cp /bin/sleep /tmp/systemd-helper && /tmp/systemd-helper 5(rename_exec.sh)	execve (59)	prism_t1036_exec_tmp	detect_t1036_masquerading → T1036.005-Masquerade	Process in temp directory claimed legitimate system name	YES	< 8s
2	T1036.005	prctl(PR_SET_NAME, "kworker/u4:2") (prctl_spoof.c)	prctl (158), a0=0xf	prism_t1036_prctl	detect_t1036_masquerading → T1036.005-Masquerade-Prctl	Process explicitly renamed to system thread via prctl	YES	< 8s
3	T1003.007	open ("/proc/223290/mem") + read (proc_mem_scan.py)	open (2) + PATH	prism_t1003_proc_open	detect_t1003_credential_d umping → T1003.007-CredentialDump	Unauthorized /proc memory access	YES	< 8s
4	T1003	open ("/etc/shadow") (shadow_access.sh)	open (2) + PATH	prism_t1003_shadow	detect_t1003_credential_d umping → T1003-ShadowAccess	Unauthorized shadow file access	YES	< 8s
5	T1036.003	Binary rename + exec from /dev/shm	execve (59)	prism_t1036_exec_s hmem	detect_t1036_masquerading → T1036.003-TempExec	Binary executed from writable temp path	YES (rule)	—
6	GAP	prctl(PR_SET_NAME) before auditd rules load	None	—	None	—	BLIND SPOT	—
7	GAP	memfd_create + fexecve (fileless execution)	memfd_create (319)	Not monitored	None	—	BLIND SPOT	—
8	GAP	ptrace-based memory access (gdb, gcore)	ptrace (101)	prism_t1003_ptrace	Not yet correlated	—	PARTIAL	—

Run Evidence Summary

Metric	Value
Total offensive actions emulated	4 (+3 gap scenarios)
Fully detected	4 (rows 1–4)
Partially detected	1 (row 8 — ptrace)
Confirmed blind spots	2 (rows 6–7)
Total alerts fired (raw engine output)	112
True positive alerts	4
False positives observed	108 (/proc/self/maps — fixed by filtering /proc/self/*)
Run latency (red script → audit log capture)	< 8s